AI-Assisted Software Engineering

Martin Kellogg

Announcements

- Check your email!
  - All teams must migrate their Covey.town from Twilio to Agora by 7 December
  - Talk to your team and your mentor about how you plan to accommodate this required migration
- Sign-ups for preliminary demo 2 are open
- Midterm 2 on 23 November
  - note: no “Your Choice Reading Quiz” on this exam
- My office hours next week are cancelled

Two seemingly-contradictory studies

[1] Becker, J., Rush, N., Barnes, E. & Rein, D. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. METR, 2025. arXiv:2507.09089
[2] Villani, T. S. & Kellogg, M. Command Coding: A Methodological Framework for AI-Accelerated Software Development. ASE 2026 (Industry Showcase). doi:10.1145/3832783.3834536

Two seemingly-contradictory studies

- METR's randomized trial [1]:
  - 16 experienced open-source developers
  - 246 tasks in repos they knew well

[1] Becker, J., Rush, N., Barnes, E. & Rein, D. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. METR, 2025. arXiv:2507.09089
[2] Villani, T. S. & Kellogg, M. Command Coding: A Methodological Framework for AI-Accelerated Software Development. ASE 2026 (Industry Showcase). doi:10.1145/3832783.3834536

Two seemingly-contradictory studies

- METR's randomized trial [1]:
  - 16 experienced open-source developers
  - 246 tasks in repos they knew well
  - With AI tools they **were 19% slower**
    - But they **believed** they were about **20% faster**.

[1] Becker, J., Rush, N., Barnes, E. & Rein, D. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. METR, 2025. arXiv:2507.09089
[2] Villani, T. S. & Kellogg, M. Command Coding: A Methodological Framework for AI-Accelerated Software Development. ASE 2026 (Industry Showcase). doi:10.1145/3832783.3834536

Two seemingly-contradictory studies

- METR's randomized trial [1]:
  - 16 experienced open-source developers
  - 246 tasks in repos they knew well
  - With AI tools they **were 19% slower**
    - But they **believed** they were about **20% faster**.
- In a case study, an NJII engineer built an 88K LoC application in 41 working days exclusively using coding agents
  - COCOMO-style estimation suggests **25-40x speedup**

[1] Becker, J., Rush, N., Barnes, E. & Rein, D. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. METR, 2025. arXiv:2507.09089
[2] Villani, T. S. & Kellogg, M. Command Coding: A Methodological Framework for AI-Accelerated Software Development. ASE 2026 (Industry Showcase). doi:10.1145/3832783.3834536

Two seemingly-contradictory studies

- METR's randomized trial [1]:
  - 16 experienced open-sourc
  - 246 tasks in repos they kne
  - With AI tools they **were 19% slower**
    - But they **believed** they were about **20% faster**.
- In a case study, an NJII engineer built an 88K LoC application in 41 working days exclusively using coding agents
  - COCOMO-style estimation suggests **25-40x speedup**

Big question for today: how can **both** of these studies be right?

[1] Becker, J., Rush, N., Barnes, E. & Rein, D. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. METR, 2025. arXiv:2507.09089
[2] Villani, T. S. & Kellogg, M. Command Coding: A Methodological Framework for AI-Accelerated Software Development. ASE 2026 (Industry Showcase). doi:10.1145/3832783.3834536

AI-Assisted SE

Today's lecture:

- how coding agents work
- best practices for interacting with coding agents
- in-class activity
- how to tell whether an agent did the right thing
- what agents might mean for SE careers

AI-Assisted SE

Today's lecture:

- **how coding agents work**
- best practices for interacting with coding agents
- in-class activity
- how to tell whether an agent did the right thing
- what agents might mean for SE careers

What does an LLM actually do?

- An LLM is function:
  - from a sequence of tokens

What does an LLM actually do?

- An LLM is function:
  - from a sequence of tokens
  - to a **probability distribution** for the next token, based on what it's seen in its training data

What does an LLM actually do?

- An LLM is function:
  - from a sequence of tokens
  - to a **probability distribution** for the next token, based on what it's seen in its training data
- The LLM then samples one of these tokens based on probability and appends it to the rest of the input sequence, and then calls itself again on the result
  - That is, it is ***autoregressive***: its outputs are part of its next input

What does an LLM actually do?

- An LLM is function:
  - from a sequence of tokens
  - to a **probability distributic** at it's seen in its training data
- The LLM then samples one of t and appends it to the rest of th itself again on the result
  - That is, it is ***autoregressive***: its outputs are part of its next input

Two important implications:

What does an LLM actually do?

- An LLM is function:
  - from a sequence of tokens
  - to a **probability distributio**
    it's seen in its training data
- The LLM then samples one of t
  and appends it to the rest of th
  itself again on the result
  - That is, it is ***autoregressive***: its outputs are part of its next input

Two important implications:

1. LLMs are optimized to produce **plausible** next tokens, not necessarily **correct** ones! (but those two often coincide…)

What does an LLM actually do?

- An LLM is function:
  - from a sequence of tokens
  - to a **probability distributio**
    it's seen in its training data
- The LLM then samples one of t
  and appends it to the rest of th
  itself again on the result
  - That is, it is ***autoregressive***: its outputs are part of its next input

Two important implications:

1. LLMs are optimized to produce **plausible** next tokens, not necessarily **correct** ones! (but those two often coincide…)
2. What's in the **context** shapes what comes out

LLMs are stateless

- From the reading: "LLMs are **stateless**: every time they execute a prompt they start from the same blank slate."

LLMs are stateless

- From the reading: "LLMs are **stateless**: every time they execute a prompt they start from the same blank slate."
  - A chat interface replays the whole conversation every turn!

LLMs are stateless

- From the reading: "LLMs are **stateless**: every time they execute a prompt they start from the same blank slate."
  - A chat interface replays the whole conversation every turn!
  - So, LLMs **don’t learn** from past mistakes in your codebase

LLMs are stateless

- From the reading: "LLMs are **stateless**: every time they execute a prompt they start from the same blank slate."
  - A chat interface replays the whole conversation every turn!
  - So, LLMs **don't learn** from past mistakes in your codebase
- Models have a ***context window***: a maximum amount of text the model can see at once

LLMs are stateless

- From the reading: "LLMs are **stateless**: every time they execute a prompt they start from the same blank slate."
  - A chat interface replays the whole conversation every turn!
  - So, LLMs **don't learn** from past mistakes in your codebase
- Models have a ***context window***: a maximum amount of text the model can see at once
  - Long contexts cost more (but there is caching)

LLMs are stateless

- From the reading: "LLMs are **stateless**: every time they execute a prompt they start from the same blank slate."
  - A chat interface replays the whole conversation every turn!
  - So, LLMs **don't learn** from past mistakes in your codebase
- Models have a ***context window***: a maximum amount of text the model can see at once
  - Long contexts cost more (but there is caching)
  - Long contexts also degrade quality
    - ***effective context window*** is frequently smaller than the advertised context window of a model

LLMs are stateless

- From the reading: "LLMs are **stateless**: every time they execute a prompt they start from the sam
  - A chat interface replays the
  - So, LLMs **don't learn** from p
- Models have a ***context window***: model can see at once
  - Long contexts cost more (but there is caching)
  - Long contexts also degrade quality
    - ***effective context window*** is frequently smaller than the advertised context window of a model

Q: What about "thinking" or "reasoning" models?

LLMs are stateless

- From the reading: "LLMs are **stateless**: every time they execute a prompt they start from the sam
  - A chat interface replays the
  - So, LLMs **don't learn** from p
- Models have a ***context window***: model can see at once
  - Long contexts cost more (but there is caching)
  - Long contexts also degrade quality
    - ***effective context window*** is frequently smaller than the advertised context window of a model

Q: What about "thinking" or "reasoning" models?
A: the same things apply, they just have a (hidden) draft area to work in before they answer

“LLMs” vs “agents”

- Willison’s definition from the reading: “***Agents*** run tools in a loop to achieve a goal.”

“LLMs” vs “agents”

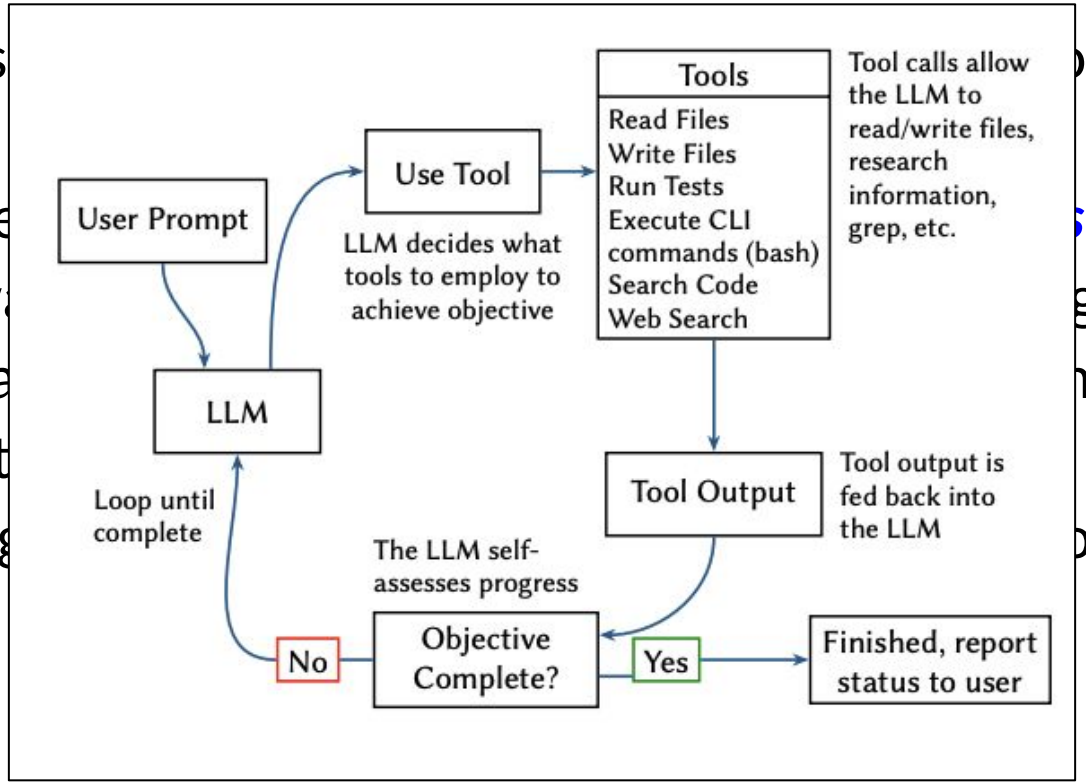
- Willison’s definition from the reading: “***Agents*** run tools in a loop to achieve a goal.”
  - an agent’s components are an LLM and a ***harness***: ordinary software that implements the loop around the agent

“LLMs” vs “agents”

- Willison’s definition from the reading: “***Agents*** run tools in a loop to achieve a goal.”
  - an agent’s components are an LLM and a ***harness***: ordinary software that implements the loop around the agent
  - the harness permits the LLM to make ***tool calls*** that allow it to use other pieces of ordinary software
    - e.g., it might read a file by calling an `open` tool

“LLMs” vs “agents”

- Willison’s
  - an age
  - the ha
    - e.g



“LLMs” vs “agents”

- Willison’s definition from the reading: “***Agents*** run tools in a loop to achieve a goal.”
  - an agent’s components are an LLM and a ***harness***: ordinary software that implements the loop around the agent
  - the harness permits the LLM to make ***tool calls*** that allow it to use other pieces of ordinary software
    - e.g., it might read a file by calling an `open` tool
- While agents are conceptually simple, production harnesses (e.g., Codex or Claude Code) are complex software systems themselves

“LLMs” vs “agents”

	chat interface	agent
sees your code?		

  op to

  t to

- .g.,

  lves

“LLMs” vs “agents”

-
-

	chat interface	agent
sees your code?	only what you paste in	reads repo, searches for what it needs
runs your code?		

“LLMs” vs “agents”

-

	chat interface	agent
sees your code?	only what you paste in	reads repo, searches for what it needs
runs your code?	no (or, at best, in a sandbox)	yes: your tests, your build, etc
who closes loop?		

-

“LLMs” vs “agents”

	chat interface	agent
sees your code?	only what you paste in	reads repo, searches for what it needs
runs your code?	no (or, at best, in a sandbox)	yes: your tests, your build, etc
who closes loop?	you (by pasting errors back)	agent (runs tests, sees output, fixes, etc)
edits files?		

  op to

  t to

- e.g.,
  lves

“LLMs” vs “agents”

	chat interface	agent
sees your code?	only what you paste in	reads repo, searches for what it needs
runs your code?	no (or, at best, in a sandbox)	yes: your tests, your build, etc
who closes loop?	you (by pasting errors back)	agent (runs tests, sees output, fixes, etc)
edits files?	you copy and paste	edits directly

  op to

  t to

- e.g.,
  lves

“LLMs” vs “agents”

	chat interface	agent
sees your code?	only wh	
runs your code?	no (or s	
who closes loop?	you (by pasting errors back)	agent (runs tests, sees output, fixes, etc)
edits files?	you copy and paste	edits directly

  op to

- e.g.,
  lves

Key difference between an agent and a “raw” LLM: the agent has a **feedback loop** - your test suite, linters, static analyses, etc.

AI-Assisted SE

Today's lecture:

- how coding agents work
- **best practices for interacting with coding agents**
- in-class activity
- how to tell whether an agent did the right thing
- what agents might mean for SE careers

Vibe coding vs agentic engineering

- ***Vibe coding***: "Accept all," don't read the diffs, paste errors back, and "forget that the code even exists."
  - can you build a production system like this? Can you build anything?

Vibe coding vs agentic engineering

- ***Vibe coding***: "Accept all," don't read the diffs, paste errors back, and "forget that the code even exists."
  - can you build a production system like this? Can you build anything? (Yes: **prototypes**!)

Vibe coding vs agentic engineering

- ***Vibe coding***: "Accept all," don't read the diffs, paste errors back, and "forget that the code even exists."
  - can you build a production system like this? Can you build anything? (Yes: **prototypes**!)
- By contrast, ***agentic engineering*** is using coding agents to build production software, where you remain responsible for its being correct, secure, maintainable, etc.

Vibe coding vs agentic engineering

- ***Vibe coding***: "Accept all," don't read the diffs, paste errors back, and "forget that the code even exists."
  - can you build a production system like this? Can you build anything? (Yes: **prototypes**!)
- By contrast, ***agentic engineering*** is using coding agents to build production software, where you remain responsible for its being correct, secure, maintainable, etc.
  - **key point**: “agentic” engineering isn’t that different than “regular” software engineering - the same techniques that help **you** build good software also help agents!

Vibe coding vs agentic engineering

- ***Vibe coding***: "Accept all," don't read the diffs, paste errors back, and "forget that the code even exists."
  - can you build a r
    anything? (Yes:
- By contrast, ***agentic***
  production softwar
  correct, secure, maintainable, etc.
  - **key point**: "agentic" engineering isn't that different than "regular" software engineering - the same techniques that help you build good software also help agents!

Quote from one of my industry collaborators: "if you couldn't lead a team of junior engineers to build a system, you can't lead a team of agents, either"

Aside: “don’t be a meat proxy”

- One of the readings was focused on why you **shouldn’t** just copy-paste the output of an LLM (agent or not) and expect a human to read it, without analyzing it yourself
  - Why not?

Aside: “don’t be a meat proxy”

- One of the readings was focused on why you **shouldn’t** just copy-paste the output of an LLM (agent or not) and expect a human to read it, without analyzing it yourself
  - Why not?
- Your value is the **understanding and judgment** you add

Aside: “don’t be a meat proxy”

- One of the readings was focused on why you **shouldn’t** just copy-paste the output of an LLM (agent or not) and expect a human to read it, without analyzing it yourself
  - Why not?
- Your value is the **understanding and judgment** you add
  - The other person could prompt the LLM themselves!

Aside: “don’t be a meat proxy”

- One of the readings was focused on why you **shouldn’t** just copy-paste the output of an LLM (agent or not) and expect a human to read it, without analyzing it yourself
  - Why not?
- Your value is the **understanding and judgment** you add
  - The other person could prompt the LLM themselves!
  - Gruhn's rule from the reading: "Read it, understand it, validate it, and then write a response **in your own words**."

“Leading a team of junior developers”

- **Quote** from one of my industry collaborators: “if you couldn’t lead a team of **junior engineers** to build a system, you can’t lead a team of agents, either”
  - How do you do that?

“Leading a team of junior developers”

- **Quote** from one of my industry collaborators: “if you couldn’t lead a team of **junior engineers** to build a system, you can’t lead a team of agents, either”
  - How do you do that?
- Also from my industrial collaborator:
  - Clear specifications
  - Careful review
  - Consistent architecture
  - Do the hard parts yourself

Command Coding

- ***Command coding*** is an AI-assisted SE methodology
  - Name from US military “Mission Command” doctrine
- Command coding has five “pillars”:

Command Coding

- ***Command coding*** is an AI-assisted SE methodology
  - Name from US military “Mission Command” doctrine
- Command coding has five “pillars”:
  - **Human-led architecture**. You define structure, patterns and security requirements before the agent writes code.

Command Coding

- ***Command coding*** is an AI-assisted SE methodology
  - Name from US military “Mission Command” doctrine
- Command coding has five “pillars”:
  - **Human-led architecture**. You define structure, patterns and security requirements before the agent writes code.
  - **Structured work decomposition**. Atomic, independently testable items with explicit acceptance criteria.

Command Coding

- ***Command coding*** is an AI-assisted SE methodology
  - Name from US military “Mission Command” doctrine
- Command coding has five “pillars”:
  - **Human-led architecture**. You define structure, patterns and security requirements before the agent writes code.
  - **Structured work decomposition**. Atomic, independently testable items with explicit acceptance criteria.
  - **Iterative multi-model review**. More on why later.

Command Coding

- ***Command coding*** is an AI-assisted SE methodology
  - Name from US military “Mission Command” doctrine
- Command coding has five “pillars”:
  - **Human-led architecture**. You define structure, patterns and security requirements before the agent writes code.
  - **Structured work decomposition**. Atomic, independently testable items with explicit acceptance criteria.
  - **Iterative multi-model review**. More on why later.
  - **Automated quality gates**. Linting, type checking, static analysis and tests on every commit. As many as you can!

Command Coding

- ***Command coding*** is an AI-assisted SE methodology
  - Name from US military “Mission Command” doctrine
- Command coding has five “pillars”:
  - **Human-led architecture**. You define structure, patterns and security requirements before the agent writes code.
  - **Structured work decomposition**. Atomic, independently testable items with explicit acceptance criteria.
  - **Iterative multi-model review**. More on why later.
  - **Automated quality gates**. Linting, type checking, static analysis and tests on every commit. As many as you can!
  - **Human oversight at decision points**. Architecture changes, security, and API design need a human decision.

Command Coding: prompt example

C.1 Implementation Prompt

Implement the following work item. Follow all established patterns exactly--do not introduce new abstractions or deviate from the existing architecture without flagging it for human review.

Work Item: [ID] -- [SHORT DESCRIPTION]

Acceptance Criteria:

- [CRITERION 1]
- [CRITERION 2]

Reference Implementation: @[PATH TO EXEMPLAR FILE] Follow this file's structure, naming conventions, options pattern, and docstring style exactly.

Relevant Context: @[ARCHITECTURE DOC OR RELATED MODULE]

Testing: [DESCRIBE HOW TO VERIFY CORRECTNESS -- e.g., sample input file, expected AST nodes, golden output, or edge cases to cover. Write tests before implementing.]

Notes: [ANY CONSTRAINTS, KNOWN GOTCHAS, OR SECURITY-SENSITIVE AREAS TO FLAG]

Begin by writing the tests. Then implement. Run the full test suite and fix any failures before considering the work item complete. Do not modify existing tests to make them pass--if a test fails, investigate whether the implementation or the test is wrong and flag ambiguous cases for human review.

Command Coding: prompt example

C.1 Implementation Prompt

Implement the following work item. Follow all established patterns exactly--do not introduce new abstractions or deviate from the existing architecture without flagging it for human review.

Work Item: [ID] -- [SHORT DESCRIPTION]

Acceptance Criteria:

- [CRITERION 1]
- [CRITERION 2]

a spec!

Reference Implementation: @[PATH TO EXEMPLAR FILE] Follow this file's structure, naming conventions, options pattern, and docstring style exactly.

Relevant Context: @[ARCHITECTURE DOC OR RELATED MODULE]

Testing: [DESCRIBE HOW TO VERIFY CORRECTNESS -- e.g., sample input file, expected AST nodes, golden output, or edge cases to cover. Write tests before implementing.]

Notes: [ANY CONSTRAINTS, KNOWN GOTCHAS, OR SECURITY-SENSITIVE AREAS TO FLAG]

Begin by writing the tests. Then implement. Run the full test suite and fix any failures before considering the work item complete. Do not modify existing tests to make them pass--if a test fails, investigate whether the implementation or the test is wrong and flag ambiguous cases for human review.

Command Coding: prompt example

C.1 Implementation Prompt

Implement the following work item. Follow all established patterns exactly--do not introduce new abstractions or deviate from the existing architecture without flagging it for human review.

Work Item: [ID] -- [SHORT DESCRIPTION]

Acceptance Criteria:

- [CRITERION 1]
- [CRITERION 2]

carefully controlled context

Reference Implementation: @[PATH TO EXEMPLAR FILE] Follow this file's structure, naming conventions, options pattern, and docstring style exactly.

Relevant Context: @[ARCHITECTURE DOC OR RELATED MODULE]

Testing: [DESCRIBE HOW TO VERIFY CORRECTNESS -- e.g., sample input file, expected AST nodes, golden output, or edge cases to cover. Write tests before implementing.]

Notes: [ANY CONSTRAINTS, KNOWN GOTCHAS, OR SECURITY-SENSITIVE AREAS TO FLAG]

Begin by writing the tests. Then implement. Run the full test suite and fix any failures before considering the work item complete. Do not modify existing tests to make them pass--if a test fails, investigate whether the implementation or the test is wrong and flag ambiguous cases for human review.

Command Coding: prompt example

C.1 Implementation Prompt

Implement the following work item. Follow all established patterns exactly--do not introduce new abstractions or deviate from the existing architecture without flagging it for human review.

Work Item: [ID] -- [SHORT DESCRIPTION]

Acceptance Criteria:

- [CRITERION 1]
- [CRITERION 2]

Reference Implementation: @[PATH TO EXEMPLAR FILE] Follow this file's structure, naming conventions, options pattern, and docstring style exactly.

Relevant Context: @[ARCHITECTURE DOC OR RELATED MODULE]

what technique is this?

Testing: [DESCRIBE HOW TO VERIFY CORRECTNESS -- e.g., sample input file, expected AST nodes, golden output, or edge cases to cover. Write tests before implementing.]

Notes: [ANY CONSTRAINTS, KNOWN GOTCHAS, OR SECURITY-SENSITIVE AREAS TO FLAG]

Begin by writing the tests. Then implement. Run the full test suite and fix any failures before considering the work item complete. Do not modify existing tests to make them pass--if a test fails, investigate whether the implementation or the test is wrong and flag ambiguous cases for human review.

Command Coding: prompt example

C.1 Implementation Prompt

Implement the following work item. Follow all established patterns exactly--do not introduce new abstractions or deviate from the existing architecture without flagging it for human review.

Work Item: [ID] -- [SHORT DESCRIPTION]

Acceptance Criteria:

- [CRITERION 1]
- [CRITERION 2]

Reference Implementation: @[PATH TO EXEMPLAR FILE] Follow this file's structure, naming conventions, options pattern, and docstring style exactly.

Relevant Context: @[ARCHITECTURE DOC OR RELATED MODULE]

Testing: [DESCRIBE HOW TO VERIFY CORRECTNESS -- e.g., sample input file, expected AST nodes, golden output, or edge cases to cover. Write tests before implementing.]

Notes: [ANY CONSTRAINTS, KNOWN GOTCHAS, OR SECURITY-SENSITIVE AREAS TO FLAG]

Begin by writing the tests. Then implement. Run the full test suite and fix any failures before considering the work item complete. Do not modify existing tests to make them pass--if a test fails, investigate whether the implementation or the test is wrong and flag ambiguous cases for human review.

gotcha: LLMs *will* do this if you let them!

Command Coding: more prompting advice

- **Constrain scope explicitly**: LLMs tend to modify other things as they go, and you have to prevent them from doing so

Command Coding: more prompting advice

- **Constrain scope explicitly**: LLMs tend to modify other things as they go, and you have to prevent them from doing so
- Have the agent **investigate before fixing**: if you just tell the LLM to fix e.g., a static analysis warning, it will treat it as a true positive
  - but is it always?

Command Coding: more prompting advice

- **Constrain scope explicitly**: LLMs tend to modify other things as they go, and you have to prevent them from doing so
- Have the agent **investigate before fixing**: if you just tell the LLM to fix e.g., a static analysis warning, it will treat it as a true positive
  - but is it always?
- Ask for **non-functional requirements** explicitly
  - E.g., agents "do not always volunteer security considerations unless explicitly prompted"

Command Coding: more prompting advice

- **Constrain scope explicitly**: LLMs tend to modify other things as they go, and you have to prevent them from doing so
- Have the agent **investigate before fixing**: if you just tell the LLM to fix e.g., a static analysis warning, it will treat it as a true positive
  - but is it always?
- Ask for **non-functional requirements** explicitly
  - E.g., agents "do not always volunteer security considerations unless explicitly prompted"
- Don't let the agent's **context fill with junk**. Periodically start over or compact context. Good planning can help!

Command Coding: more prompting advice

- **Constrain scope explicitly**: LLMs tend to modify other things as they go, and you have to prevent them from doing so
- Have the agent **investi**
  LLM to fix e.g., a sta
  positive
  - but is it always?
- Ask for **non-functio**
  - E.g., agents "do
    considerations
- Don't let the agent's **context fill with junk**. Periodically start over or compact context. Good planning can help!

What about `AGENTS.md` or similar files? Some recent work (Gloaguen et al., from ETH Zurich) suggests they increase cost but **don't** improve success rate.

Command Coding: more prompting advice

- **Constrain scope explicitly**: LLMs tend to modify other things as they go, and you have to prevent them from doing so
- Have the agent **investi**
  LLM to fix e.g., a sta
  positive
  - but is it always?
- Ask for **non-functio**
  - E.g., agents "do
    considerations
- Don't let the agent's **context fill with junk**. Periodically start over or compact context. Good planning can help!

What about `AGENTS.md` or similar files? Some recent work (Gloaguen et al., from ETH Zurich) suggests they increase cost but **don't** improve success rate. Implication: maybe it's a **show don't tell** situation…

CC: Contextual Quality Contagion

- Because code is always part of a coding agent's context, **quality signals early** in the project are very important

CC: Contextual Quality Contagion

- Because code is always part of a coding agent's context, **quality signals early** in the project are very important
  - If the existing code looks like it follows good engineering practices, new agent-written code is more likely to, too

CC: Contextual Quality Contagion

- Because code is always part of a coding agent's context, **quality signals early** in the project are very important
  - If the existing code looks like it follows good engineering practices, new agent-written code is more likely to, too
  - We called this the ***contextual quality contagion hypothesis***
    - we haven't proven it yet!

CC: Contextual Quality Contagion

- Because code is always part of a coding agent's context, **quality signals early** in the project are very important
  - If the existing code looks like it follows good engineering practices, new agent-written code is more likely to, too
  - We called this the ***contextual quality contagion hypothesis***
    - we haven't proven it yet!
- **Quality gates** (static analysis, good testing practices, etc) do double duty:
  - not only do they help you know the agent did a good job
  - but they also signal good engineering discipline to the agent

Command Coding: Multi-modal Review

- **Multi-modal review** was one of the pillars of command coding
  - one model (e.g., Claude) generates code; another (e.g., Gemini) does the review

Command Coding: Multi-modal Review

- **Multi-modal review** was one of the pillars of command coding
  - one model (e.g., Claude) generates code; another (e.g., Gemini) does the review
  - we found cases where one LLM said “LGTM” to its own code, but another model found e.g., a **security vulnerability** (!)

Command Coding: Multi-modal Review

- **Multi-modal review** was one of the pillars of command coding
  - one model (e.g., Claude) generates code; another (e.g., Gemini) does the review
  - we found cases where one LLM said “LGTM” to its own code, but another model found e.g., a **security vulnerability** (!)
- Evidence suggests that LLMs **prefer their own writing**

Command Coding: Multi-modal Review

- **Multi-modal review** was one of the pillars of command coding
  - one model (e.g., Claude) generates code; another (e.g., Gemini) does the review
  - we found cases where one LLM said “LGTM” to its own code, but another model found e.g., a **security vulnerability** (!)
- Evidence suggests that LLMs **prefer their own writing**
  - or writing from other models in their own ***model family*** (e.g., Claude Sonnet prefers Claude Haiku’s writing to GPT’s)

Command Coding: Multi-modal Review

- **Multi-modal review** was one of the pillars of command coding
  - one model (e.g., Claude) generates code; another (e.g., Gemini) does the review
  - we found cases where one LLM said “LGTM” to its own code, but another model found e.g., a **security vulnerability** (!)
- Evidence suggests that LLMs **prefer their own writing**
  - or writing from other models in their own ***model family*** (e.g., Claude Sonnet prefers Claude Haiku’s writing to GPT’s)
- Why might this be so? A model finds it own output to be **more plausible**, almost by definition

AI-Assisted SE

Today's lecture:

- how coding agents work
- best practices for interacting with coding agents
- **in-class activity**
- how to tell whether an agent did the right thing
- what agents might mean for SE careers

In-class activity: an unreasonable task

- Form groups with 3 or 4 nearby students
  - Each group should nominate one “driver” based on who thinks they have access to the **fanciest AI tooling**
- You have 10 minutes from now to build code to the spec and test suite that the QR code on this page links to
  - this is intentionally **not plausible** if you program by hand
- I have a hidden test suite that I’ll distribute after ten minutes, and we’ll see how you did.



or: https://tinyurl.com/bdz735hm

In-class activity: how’d it go?

- Time’s up! Who passed all the public tests?
- Download the hidden tests with this new QR code
  - Raise your hand once you’re done checking



or: https://tinyurl.com/mwafpvzf

In-class activity: how'd it go?

- Time's up! Who passed all the public tests?
- Download the hidden tests with this new QR code
  - Raise your hand once you're done checking



What went well or poorly?

or: https://tinyurl.com/mwafpvzf

AI-Assisted SE

Today's lecture:

- how coding agents work
- best practices for interacting with coding agents
- in-class activity
- **how to tell whether an agent did the right thing**
- what agents might mean for SE careers

Verification and Validation

- Did you know if the code that your LLM wrote in the activity was **right**? Why or why not?
  - Did you even have time to read it?
  - Would you have shipped such code to production?

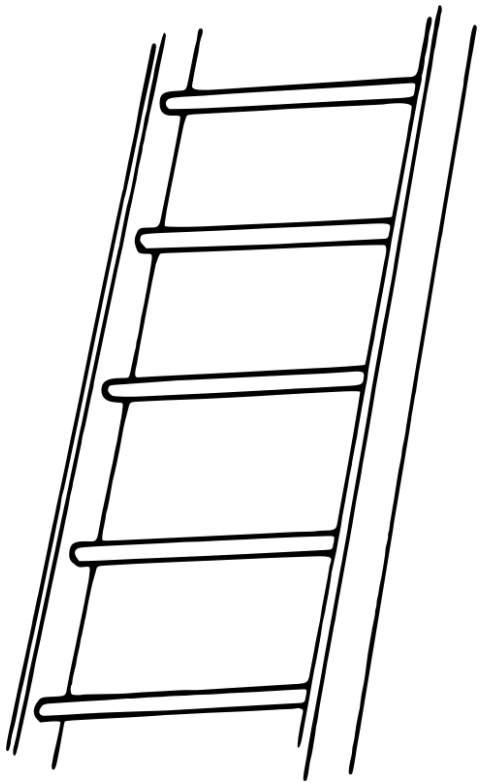
Verification and Validation

- Did you know if the code that your LLM wrote in the activity was **right**? Why or why not?
  - Did you even have time to read it?
  - Would you have shipped such code to production?
- LLMs can produce code **very quickly** compared to human programmers!
  - But the question of “is it right?” is still on us

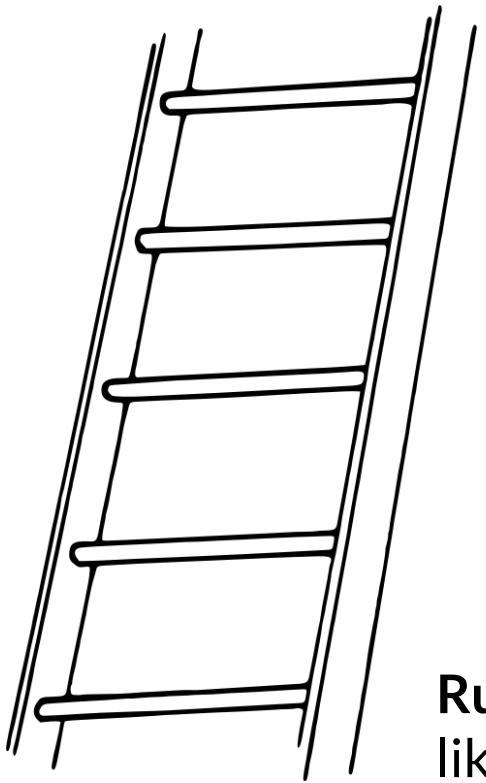
Verification and Validation

- Did you know if the code that your LLM wrote in the activity was **right**? Why or why not?
  - Did you even have time to read it?
  - Would you have shipped such code to production?
- LLMs can produce code **very quickly** compared to human programmers!
  - But the question of “is it right?” is still on us
  - And, with coding agents writing more code, it’s **more urgent than ever**

Ladder of Correctness

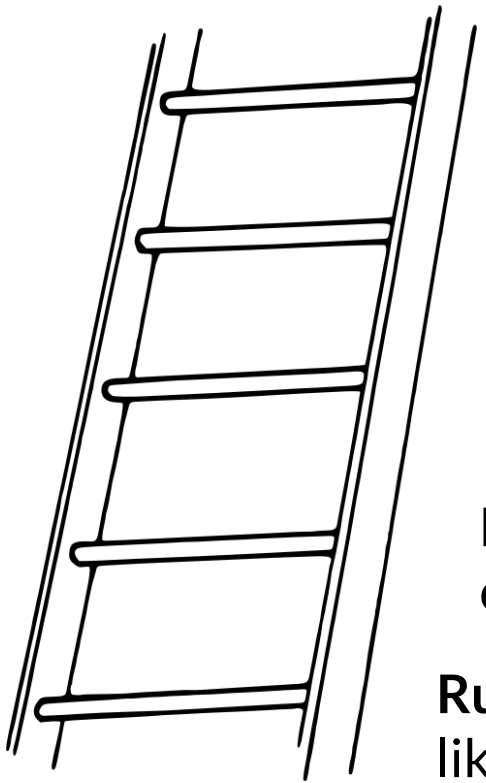


Ladder of Correctness



Rung 0: “The AI said it works.” Not evidence - you’re acting like a “**meat proxy**” from the reading.

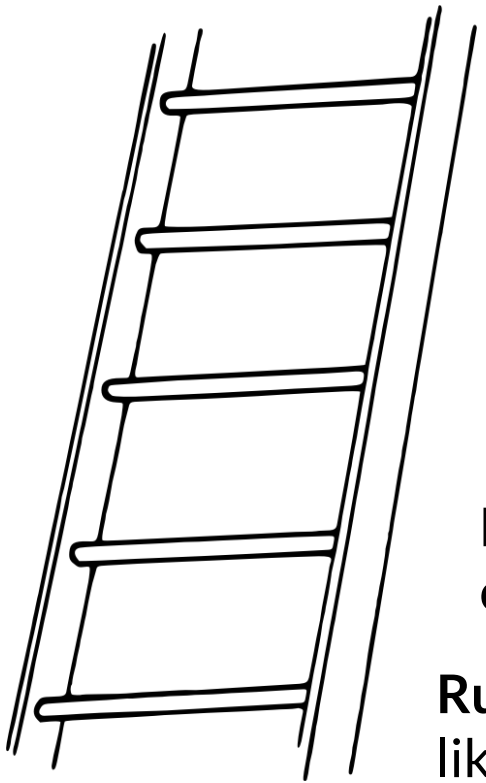
Ladder of Correctness



Rung 1: “The tests pass.” Who wrote the tests? Do they check the right thing? Could they **be gamed**?

Rung 0: “The AI said it works.” Not evidence - you’re acting like a “**meat proxy**” from the reading.

Ladder of Correctness

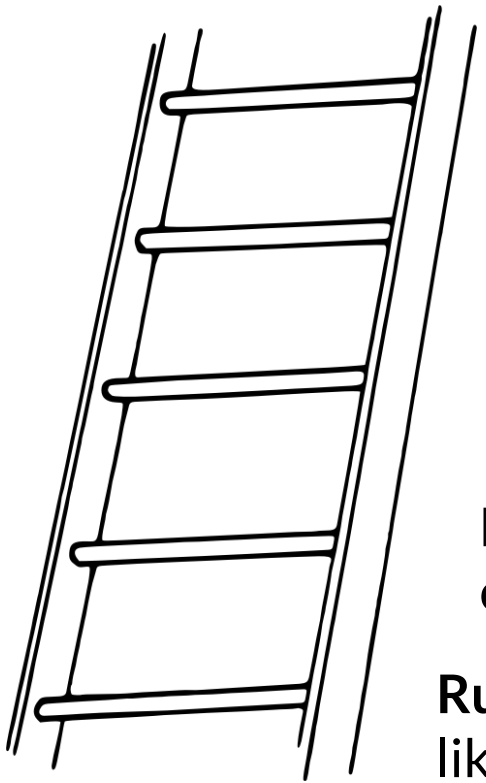


Rung 2: “The tests are good.” Reviewed oracles, coverage, mutation score. Still only what we **thought to test**.

Rung 1: “The tests pass.” Who wrote the tests? Do they check the right thing? Could they **be gamed**?

Rung 0: “The AI said it works.” Not evidence - you’re acting like a “**meat proxy**” from the reading.

Ladder of Correctness



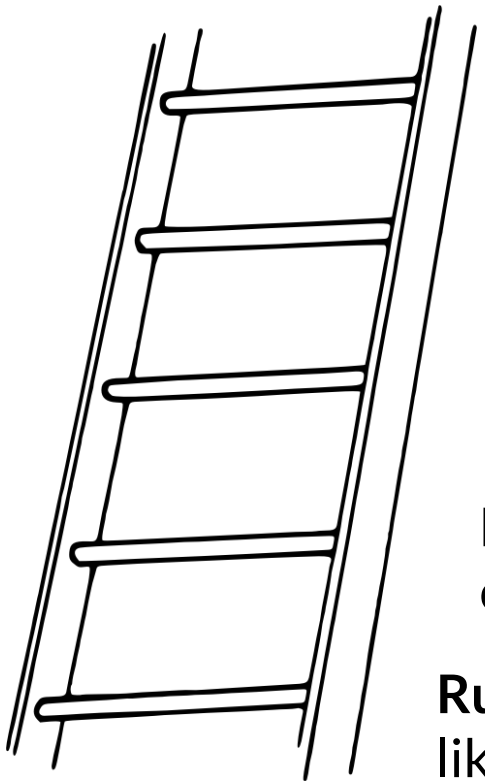
Rung 3: “Whole classes of bugs are impossible.” Types, **sound** static analysis. Only the analyzed properties.

Rung 2: “The tests are good.” Reviewed oracles, coverage, mutation score. Still only what we **thought to test**.

Rung 1: “The tests pass.” Who wrote the tests? Do they check the right thing? Could they **be gamed**?

Rung 0: “The AI said it works.” Not evidence - you’re acting like a “**meat proxy**” from the reading.

Ladder of Correctness



Rung 4: “It’s proven to meet the spec.” Full formal verification. Problem: how do you know the **spec is right**?

Rung 3: “Whole classes of bugs are impossible.” Types, **sound** static analysis. Only the analyzed properties.

Rung 2: “The tests are good.” Reviewed oracles, coverage, mutation score. Still only what we **thought to test**.

Rung 1: “The tests pass.” Who wrote the tests? Do they check the right thing? Could they **be gamed**?

Rung 0: “The AI said it works.” Not evidence - you’re acting like a “**meat proxy**” from the reading.

Ladder of Correctness



Rung 4: “It’s proven to meet the spec.” Full formal verification. Problem: how do you know the **spec is right**?

Rung 3: “Whole classes of bugs are impossible.” Types, **sound** static analysis. Only the analyzed properties.

Rung 2: “The tests are good.” Reviewed oracles, coverage, mutation score. Still only what we **thought to test**.

My claim: agents both **require** and **enable** us to move higher on this ladder.

e tests pass.” Who wrote the tests? Do they
ght thing? Could they **be gamed**?

AI said it works.” Not evidence - you’re acting
like a “**meat proxy**” from the reading.

Rung 1: can we trust agent-written tests?

- A recent paper introduced the **ImpossibleBench**
  - Took coding benchmarks and changed the tests so that they specifically contradicted the specification.
  - Asked model to implement spec and pass tests

[1] Zhong, Z., Raghunathan, A. & Carlini, N. *ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases.* ICLR 2026.

Rung 1: can we trust agent-written tests?

- A recent paper introduced the **ImpossibleBench**
  - Took coding benchmarks and changed the tests so that they specifically contradicted the specification.
  - Asked model to implement spec and pass tests
  - Any passing solution therefore **must have cheated**!

[1] Zhong, Z., Raghunathan, A. & Carlini, N. *ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases.* ICLR 2026.

Rung 1: can we trust agent-written tests?

- A recent paper introduced the **ImpossibleBench**
  - Took coding benchmarks and changed the tests so that they specifically contradicted the specification.
  - Asked model to implement spec and pass tests
  - Any passing solution therefore **must have cheated**!
- Frontier models **cheat often**: e.g., GPT-5 cheated on 54% of tested tasks in one of their benchmarks
  - LLMs will try to “make the tests green” by any means necessary
    - e.g., changing oracles, special-casing test inputs, etc

[1] Zhong, Z., Raghunathan, A. & Carlini, N. *ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases.* ICLR 2026.

Rung 1: can we trust agent-written tests?

- A recent paper introduced the **ImpossibleBench**
  - Took coding ben[illegible]hat they specifically contr[illegible]
  - Asked model to i[illegible]
  - Any passing solution therefore **must have cheated**!
- Frontier models **cheat often**: e.g., GPT-5 cheated on 54% of tested tasks in one of their benchmarks
  - LLMs will try to “make the tests green” by any means necessary
    - e.g., changing oracles, special-casing test inputs, etc

Implication: an agent will **optimize the target you give it**

[1] Zhong, Z., Raghunathan, A. & Carlini, N. *ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases.* ICLR 2026.

Rung 1 done well: TDFlow

- Recall **test-driven development** from our testing lectures?

Han, K., Maddikayala, S., Knappe, T., Patel, O., Liao, A. & Barati Farimani, A. TDFlow: Agentic Workflows for Test Driven Development. EACL 2026. arXiv:2510.23761

Rung 1 done well: TDFlow

- Recall **test-driven development** from our testing lectures?
- Another recent paper found that when **human write tests** (and agents aren't allowed to change them), actual success rate on tested tasks goes from ~68% to ~94%
  - that's a role for **you**: decide what the system **should do**

Han, K., Maddikayala, S., Knappe, T., Patel, O., Liao, A. & Barati Farimani, A. TDFlow: Agentic Workflows for Test Driven Development. EACL 2026. arXiv:2510.23761

Rung 1 done well: TDFlow

- Recall **test-driven development** from our testing lectures?
- Another recent paper found that when **human write tests** (and agents aren't allowed to change them), actual success rate on tested tasks goes from ~68% to ~94%
  - that's a role for **you**: decide what the system **should do**
- Effectively, you can think of this as "making the agents do TDD" (with tests that you supply)

Han, K., Maddikayala, S., Knappe, T., Patel, O., Liao, A. & Barati Farimani, A. TDFlow: Agentic Workflows for Test Driven Development. EACL 2026. arXiv:2510.23761

Rung 2: quis custodiet ipsos custodies

- Our experience in the industrial case study mentioned earlier:
  - AI-generated tests sometimes "checked that code **did what it did rather than what it should do**”
  - common failure modes: tautological oracles, excessive mocking

Rung 2: quis custodiet ipsos custodies

- Our experience in the industrial case study mentioned earlier:
  - AI-generated tests sometimes "checked that code **did what it did rather than what it should do**”
  - common failure modes: tautological oracles, excessive mocking
- Need **test suite quality metrics**

Rung 2: quis custodiet ipsos custodies

- Our experience in the industrial case study mentioned earlier:
  - AI-generated tests sometimes "checked that code **did what it did rather than what it should do**”
  - common failure modes: tautological oracles, excessive mocking
- Need **test suite quality metrics**
  - Mutation testing (for oracles), coverage (for inputs)

Rung 2: quis custodiet ipsos custodies

- Our experience in the industrial case study mentioned earlier:
  - AI-generated tests sometimes "checked that code **did what it did rather than what it should do**”
  - common failure modes: tautological oracles, excessive mocking
- Need **test suite quality metrics**
  - Mutation testing (for oracles), coverage (for inputs)
- Mutation score works as a **fitness function** for an agent
  - gives the agent a measurable target that's much harder to game than "tests pass"

Rung 3: types and static analysis

- Key property is **soundness**
  - why?

Rung 3: types and static analysis

- Key property is **soundness**
  - why? Guarantees the LLM **cannot cheat**

Rung 3: types and static analysis

- Key property is **soundness**
  - why? Guarantees the LLM **cannot cheat**
- Lots of evidence that this helps, including:
  - GitHub blog: "*Why AI is pushing developers toward typed languages*" (2025)
  - Our industrial case study: 100% type coverage from day one (in Python). Caught hallucinated library signatures, for example.

Rung 3: types and static analysis

- Key property is **soundness**
  - why? Guarantees the LLM **cannot cheat**
- Lots of evidence that this helps, including:
  - GitHub blog: "*Why AI is pushing developers toward typed languages*" (2025)
  - Our industrial case study: 100% type coverage from day one (in Python). Caught hallucinated library signatures, for example.
- Danger: LLMs can make the analysis happy by **breaking something it doesn't check**. I saw this in my own work (ISSTA 2026): LLMs will make the checker pass by breaking something else!

Rung 4: full formal verification

- Verifiers are the **strongest static analyzers** possible
  - historically been too expensive; 20:1 proof:code ratio in seL4

Rung 4: full formal verification

- Verifiers are the **strongest static analyzers** possible
  - historically been too expensive; 20:1 proof:code ratio in seL4
- Possible future: the LLM proposes code and proofs, the verifier checks them, the **human checks only the spec**

Rung 4: full formal verification

- Verifiers are the **strongest static analyzers** possible
  - historically been too expensive; 20:1 proof:code ratio in seL4
- Possible future: the LLM proposes code and proofs, the verifier checks them, the **human checks only the spec**
  - unlike in math, we don't care about *how* the proof was done, just that it goes through

Rung 4: full formal verification

- Verifiers are the **strongest static analyzers** possible
  - historically been too expensive; 20:1 proof:code ratio in seL4
- Possible future: the LLM proposes code and proofs, the verifier checks them, the **human checks only the spec**
  - unlike in math, we don't care about *how* the proof was done, just that it goes through
  - human job moves **up a level of abstraction**: checking the correctness of a 20-line spec instead of a 1,000 line implementation is a better use of **limited human attention**

Rung 4: full formal verification

- Verifiers are the **strongest static analyzers** possible
  - historically been too expensive; 20:1 proof:code ratio in seL4
- Possible future: the LLM proposes code and proofs, the verifier checks them, the **human checks only the spec**
  - unlike in math, we don't care about *how* the proof was done, just that it goes through
  - human job moves **up a level of abstraction**: checking the correctness of a 20-line spec instead of a 1,000 line implementation is a better use of **limited human attention**
  - time for you to learn PL? Ask Dr Kellogg in OH if you're curious

Rung 4: full formal verification

- Verifiers are the **strongest static analyzers** possible
  - historically been too expensive; 20:1 proof:code ratio in seL4
- Possible future: the LLM proposes code and proofs, the verifier checks them, the **human checks only the spec**
  - unlike in math, we don't care about *how* the proof was done,
  - g the
  - tention
  - time for you to learn PL: Ask DJ Kellogg in OH if you're curious

Anthropic recently (22 October 2026) put out a blog post about how "LLMs are making formal software verification feasible at a much larger scale": https://www.anthropic.com/research/formal-verification

AI-Assisted SE

Today's lecture:

- how coding agents work
- best practices for interacting with coding agents
- in-class activity
- how to tell whether an agent did the right thing
- **what agents might mean for SE careers**

Coding agents and you

- There is good evidence that coding agents are dramatically changing software engineering

Coding agents and you

- There is good evidence that coding agents are dramatically changing software engineering
  - e.g., DORA 2025 showed **90%** of surveyed professional developers were already using AI for coding

Coding agents and you

- There is good evidence that coding agents are dramatically changing software engineering
  - e.g., DORA 2025 showed **90%** of surveyed professional developers were already using AI for coding
  - Stack Overflow developer survey 2025 said that ~⅔ of developers are **frustrated with AI output**, though
    - verification and validation remain critical human activities

Coding agents and you

- There is good evidence that coding agents are dramatically changing software engineering
  - e.g., DORA 2025 showed **90%** of surveyed professional developers were already using AI for coding
  - Stack Overflow developer survey 2025 said that ~⅔ of developers are **frustrated with AI output**, though
    - verification and validation remain critical human activities
- Job market for new grads right now is softer than 5-10 years ago
  - possibly because AI has replaced some junior engineers
  - but also higher interest rates, overhiring during covid, etc.

Coding agents and you

- All that said, there is still a role for humans to play in software engineering. Maybe **less writing code**, but more:

Coding agents and you

- All that said, there is still a role for humans to play in software engineering. Maybe **less writing code**, but more:
  - Deciding what to build, and weighing **trade-offs**
  - **Specifying** it precisely enough for an agent to build it
  - Designing **architecture** worth copying
  - Verifying that it's right, and **convincing others** it's right
  - **Taking responsibility** for it (“a computer must never make a management decision…”)

Coding agents and you

- All that said, there is still a role for humans to play in software engineering. Maybe **less writing code**, but more:
  - Deciding what to build, and weighing **trade-offs**
  - **Specifying** it precisely enough for an agent to build it
  - Designing **architecture** worth copying
  - Verifying that it's right, and **convincing others** it's right
  - **Taking responsibility** f… …ke a management decision.

In what I'm sure is a surprising turn of events, we've spent most of this semester on **exactly these things**

AI-Assisted SE: takeaways

- Coding agents put an LLM in a feedback loop with the ability to call regular software (“tools”), like your test suite
- LLMs produce plausible completions, given their context
  - Control the context to control the agent’s output quality
- Don't let a model grade its own work. Don’t be a meat proxy.
- Agents make verification both more necessary and more affordable. Climb the ladder.
- “if you couldn't effectively lead a team of junior developers building a system, you likely cannot effectively command AI agents to build it”: the **bar is higher**, so hold yourself up to it